FOUNDATIONS OF IoT

STUDENT REPOSITORY EVALUATION REPORT

Repository: 25011102113_FIoT_LAB

OVERALL STATUS: PASSED

Evaluation Summary

Required CAT Folders	Projects Discovered
14	29
Projects Passed	**Projects Failed**
29	0
Missing CAT Folders	**CATs Without Projects**
0	0
File Checks Passed	
206 / 206	

Platform-wise Results

Platform	Tasks	Passed	Failed
8051	13	13	0
STM32	16	16	0

The following pages contain the detailed task evaluation, file checks and README content.

Platform: 8051

8051/CAT2/TASK2B

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT2/TASK2B: Software-delay LED blink

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine software-delay led blink on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0 | Active-low LED through series resistor |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

The LED alternates on and off. Software-loop timing is uncalibrated.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT2/TASK2B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

The LED alternates on and off. Software-loop timing is uncalibrated.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT3/TASK3A

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT3/TASK3A: Debounced push-button LED toggle

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

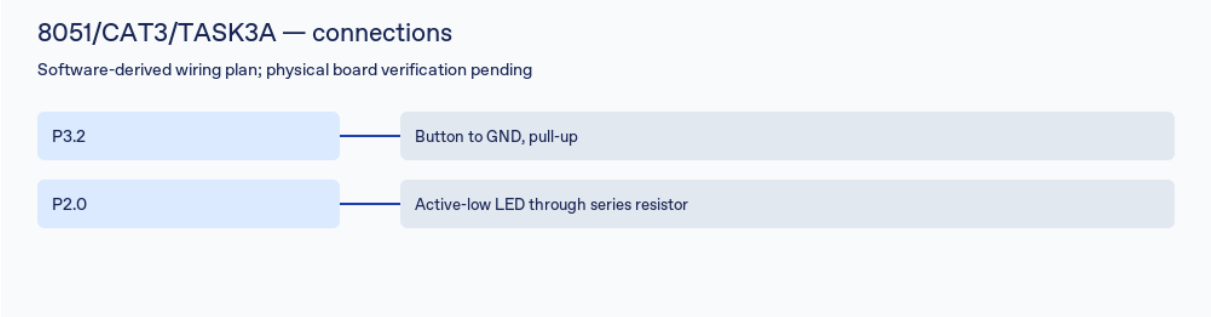
Implement and examine debounced push-button led toggle on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P3.2 | Button to GND, pull-up |

| P2.0 | Active-low LED through series resistor |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

A debounced button press toggles the LED once.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT3/TASK3A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

A debounced button press toggles the LED once.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT4/TASK4A

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51

- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT4/TASK4A: Matrix keypad and seven-segment interface

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine matrix keypad and seven-segment interface on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0-P2.3 | Keypad rows |

| P2.4-P2.7 | Keypad columns |

| P0.0-P0.7 | Seven-segment a-g/dp via buffered driver and resistors |

| P1.0-P1.3 | Active-low LEDs via series resistors |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Numeric keys display digits and select the implemented LED patterns.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT4/TASK4A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Numeric keys display digits and select the implemented LED patterns.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT5/TASK5A

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT5/TASK5A: Timer0 interrupt time base

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine timer0 interrupt time base on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P1.0 | Active-low LED through series resistor |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

The LED toggles after 1000 nominal millisecond timer interrupts.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT5/TASK5A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

The LED toggles after 1000 nominal millisecond timer interrupts.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT5/TASK5B/TASK5B.1

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT5/TASK5B/TASK5B.1: Timer-driven software PWM fading

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

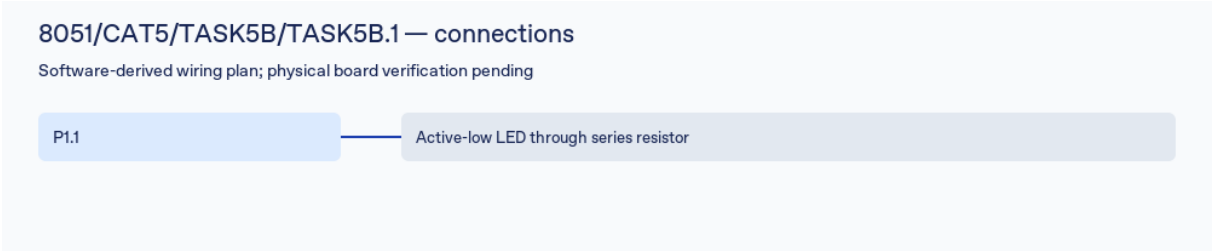
Implement and examine timer-driven software pwm fading on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P1.1 | Active-low LED through series resistor |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Duty cycles rise from 0% to 100% in 10% increments and fall back; nominal carrier is 500 Hz before ISR latency.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT5/TASK5B/TASK5B.1

See [toolchain setup and build limits](../../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Duty cycles rise from 0% to 100% in 10% increments and fall back; nominal carrier is 500 Hz before ISR latency.

Actual verified results

See [recorded build and file checks](../../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT5/TASK5B/TASK5B.2

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT5/TASK5B/TASK5B.2: Timer2 variable-frequency hardware clock output

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

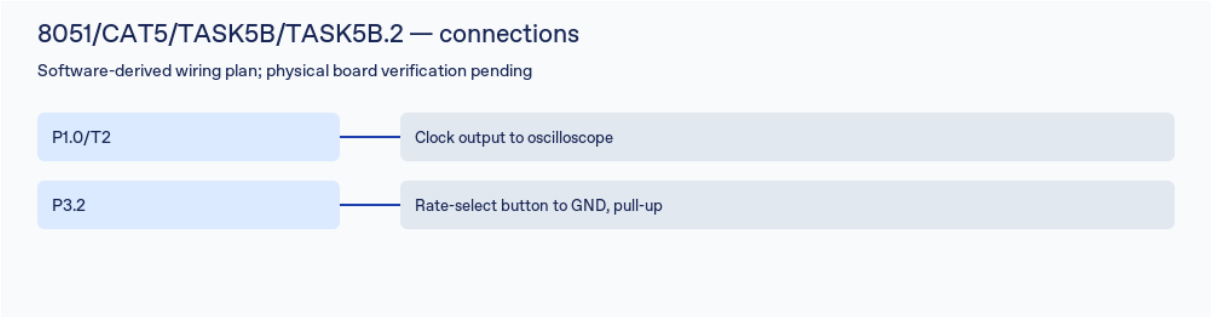
Implement and examine timer2 variable-frequency hardware clock output on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P1.0/T2 | Clock output to oscilloscope |

| P3.2 | Rate-select button to GND, pull-up |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Each debounced press selects the next nominal rate: 100, 500, 1000, 2000 or 5000 Hz, with approximately 50% duty.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT5/TASK5B/TASK5B.2

See [toolchain setup and build limits](../../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Each debounced press selects the next nominal rate: 100, 500, 1000, 2000 or 5000 Hz, with approximately 50% duty.

Actual verified results

See [recorded build and file checks](../../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT5/TASK5B/TASK5B.3

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT5/TASK5B/TASK5B.3: Timer-driven multi-LED fading

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

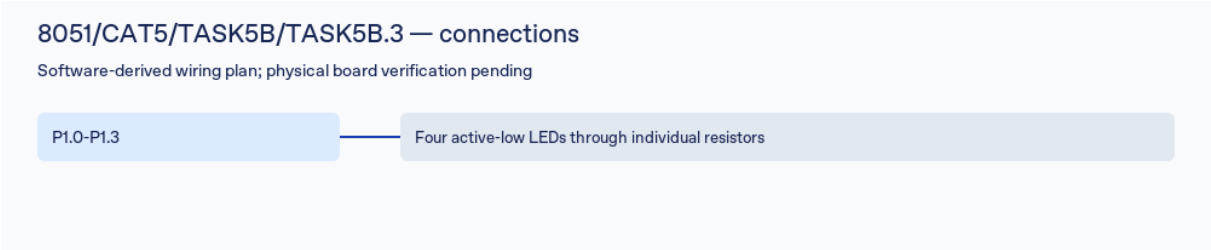
Implement and examine timer-driven multi-led fading on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P1.0-P1.3 | Four active-low LEDs through individual resistors |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Four LEDs share the timer-driven PWM breathing pattern.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT5/TASK5B/TASK5B.3

See [toolchain setup and build limits](../../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Four LEDs share the timer-driven PWM breathing pattern.

Actual verified results

See [recorded build and file checks](../../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT5/TASK5C

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex

- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT5/TASK5C: Timer-driven stepper speed and direction

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine timer-driven stepper speed and direction on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0-P2.3 | ULN2003 inputs IN1-IN4 |

| P3.2 | Direction selector to GND |

| P3.3 | Speed selector to GND |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Motor steps in the selected direction with nominal 10 ms or 100 ms step spacing.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT5/TASK5C

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Motor steps in the selected direction with nominal 10 ms or 100 ms step spacing.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT6/TASK6A

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT6/TASK6A: UART string transmission

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

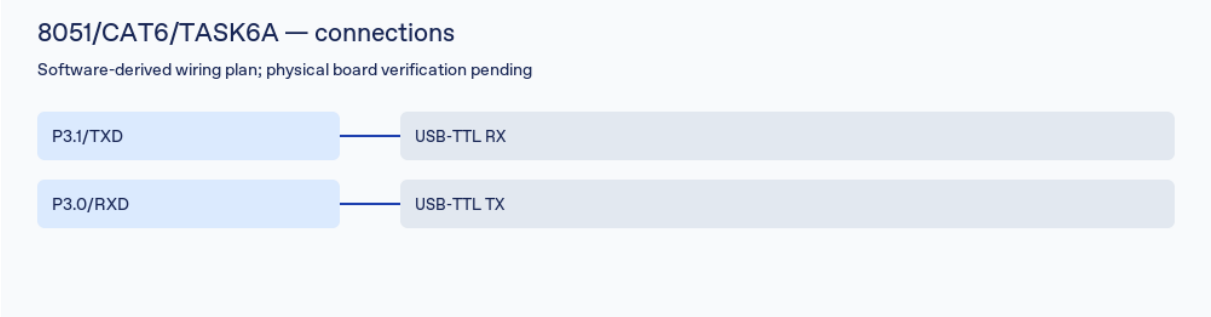
Implement and examine uart string transmission on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P3.1/TXD | USB-TTL RX |

| P3.0/RXD | USB-TTL TX |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

The terminal receives the greeting repeatedly at 9600-8-N-1.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT6/TASK6A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

The terminal receives the greeting repeatedly at 9600-8-N-1.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT6/TASK6B

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT6/TASK6B: Interrupt-driven buffered UART echo

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

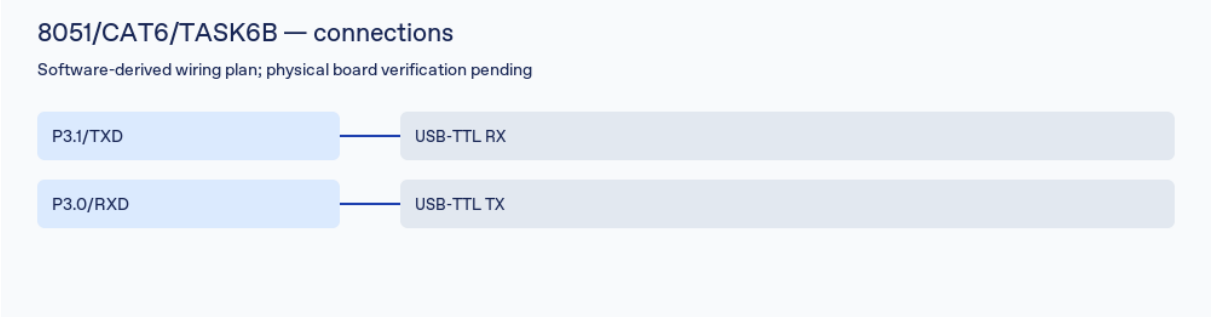
Implement and examine interrupt-driven buffered uart echo on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P3.1/TXD | USB-TTL RX |

| P3.0/RXD | USB-TTL TX |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Typed bytes echo unchanged at 9600-8-N-1. The 31-byte usable RX queue drops new bytes on overflow; rx_dropped exposes the count.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT6/TASK6B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Typed bytes echo unchanged at 9600-8-N-1. The 31-byte usable RX queue drops new bytes on overflow; rx_dropped exposes the count.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT7/TASK7A

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT7/TASK7A: LCD four-bit initialization and text

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

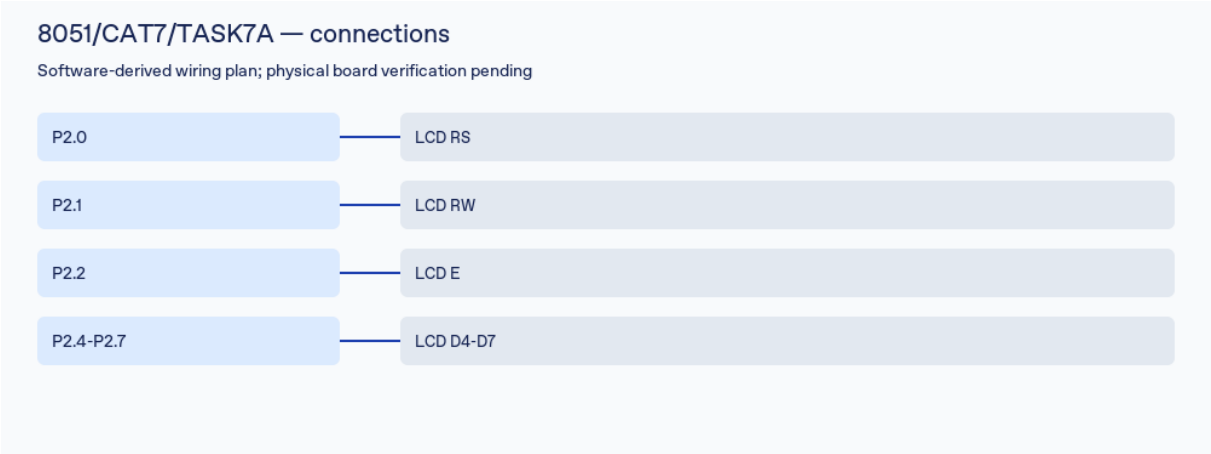
Implement and examine lcd four-bit initialization and text on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0 | LCD RS |

| P2.1 | LCD RW |

| P2.2 | LCD E |

| P2.4-P2.7 | LCD D4-D7 |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

The LCD shows the fixed text programmed in main.c.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT7/TASK7A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

The LCD shows the fixed text programmed in main.c.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT7/TASK7B

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT7/TASK7B: LCD cursor positioning and numeric update

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine lcd cursor positioning and numeric update on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0 | LCD RS |

| P2.1 | LCD RW |

| P2.2 | LCD E |

| P2.4-P2.7 | LCD D4-D7 |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

The display updates the counter at its configured cursor location.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT7/TASK7B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

The display updates the counter at its configured cursor location.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

8051/CAT7/TASK7C

File checks

- [PASS] Keil project (*.uvproj or *.uvprojx)
- [PASS] C source file (*.c)
- [PASS] STARTUP.A51
- [PASS] Objects/ folder
- [PASS] Objects/*.hex
- [PASS] README.md exists.

Task result: PASSED

README content

8051/CAT7/TASK7C: Keypad-controlled LCD character movement

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

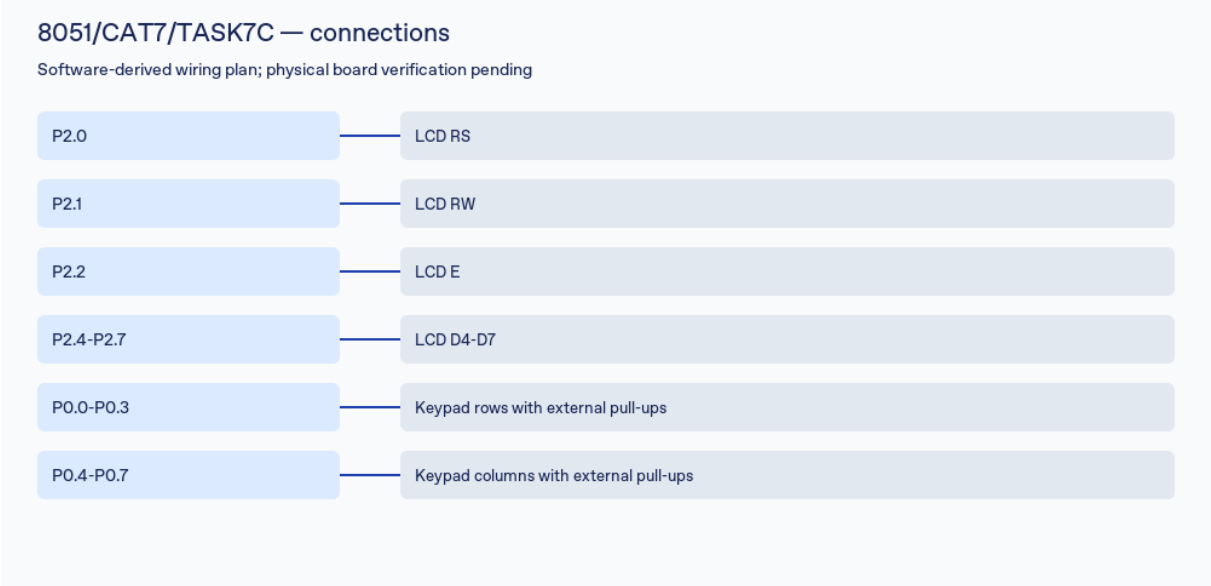
Implement and examine keypad-controlled lcd character movement on AT89S52, classic 12-clock 8051, 11.0592 MHz.

Components required and hardware platform

AT89S52 board, 11.0592 MHz crystal, regulated 5 V supply, programmer, and the peripherals listed below.

VCC pin 40 to regulated 5 V; GND pin 20; EA pin 31 HIGH for internal Flash. Use the board crystal and an active-HIGH reset pulse with RST normally LOW. A bare-chip reset uses a resistor from RST to GND and a capacitor from VCC to RST. Verify against the board schematic.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| P2.0 | LCD RS |

| P2.1 | LCD RW |

| P2.2 | LCD E |

| P2.4-P2.7 | LCD D4-D7 |

| P0.0-P0.3 | Keypad rows with external pull-ups |

| P0.4-P0.7 | Keypad columns with external pull-ups |

Use individual current-limiting resistors for LEDs and seven-segment segments. Port 0 on the 8051 needs external pull-ups; use a suitable display driver rather than exceeding GPIO current ratings. Never drive motor coils directly from an MCU; use a ULN2003 and the motor-rated supply with common ground.

Working principle and implementation

Keys 2/8/4/6 move a star up/down/left/right within the 2x16 display.

Read the complete [application source](main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py 8051/CAT7/TASK7C

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. SDCC builds a mechanically syntax-translated copy; its own startup is used. The included Keil C51 startup and project have not been executed in Keil.

Expected output

Keys 2/8/4/6 move a star up/down/left/right within the 2x16 display.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Platform: STM32

STM32/CAT10/TASK10A

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT10/TASK10A: STM32 and ESP8266 UART bridge

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine stm32 and esp8266 uart bridge on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PA9/USART1_TX | ESP8266 RX, 3.3 V logic, 115200 baud |

| PA10/USART1_RX | ESP8266 TX, 3.3 V logic, 115200 baud |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

The PC can exchange AT commands and responses with the ESP8266 over the two UARTs.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT10/TASK10A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

The PC can exchange AT commands and responses with the ESP8266 over the two UARTs.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT10/TASK10B

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT10/TASK10B: ESP8266 station-mode provisioning

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine esp8266 station-mode provisioning on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PA9/USART1_TX | ESP8266 RX, 3.3 V logic, 115200 baud |

| PA10/USART1_RX | ESP8266 TX, 3.3 V logic, 115200 baud |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

After local credentials are configured, the modem receives station-mode and connection commands. Actual connection success must be checked in responses.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT10/TASK10B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

After local credentials are configured, the modem receives station-mode and connection commands. Actual connection success must be checked in responses.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope

measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Wi-Fi credential strings are placeholders. Configure credentials locally and keep those changes uncommitted. AT-command state/error recovery and request framing remain subject to integration testing.

STM32/CAT10/TASK10C

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT10/TASK10C: ESP8266 Wi-Fi connection and address query

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine esp8266 wi-fi connection and address query on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PA9/USART1_TX | ESP8266 RX, 3.3 V logic, 115200 baud |

| PA10/USART1_RX | ESP8266 TX, 3.3 V logic, 115200 baud |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Modem AT responses are forwarded to the terminal; inspect the actual connection and address replies.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT10/TASK10C

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Modem AT responses are forwarded to the terminal; inspect the actual connection and address replies.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Wi-Fi credential strings are placeholders. Configure credentials locally and keep those changes uncommitted. AT-command state/error recovery and request framing remain subject to integration testing.

STM32/CAT11/TASK11A

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT11/TASK11A: ESP8266 TCP server setup

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine esp8266 tcp server setup on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PA9/USART1_TX | ESP8266 RX, 3.3 V logic, 115200 baud |

| PA10/USART1_RX | ESP8266 TX, 3.3 V logic, 115200 baud |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

AT commands configure an ESP8266 TCP server after a successful network connection.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT11/TASK11A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

AT commands configure an ESP8266 TCP server after a successful network connection.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Wi-Fi credential strings are placeholders. Configure credentials locally and keep those changes uncommitted. AT-command state/error recovery and request framing remain subject to integration testing.

STM32/CAT11/TASK11B

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s

- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT11/TASK11B: ESP8266 HTTP response server

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

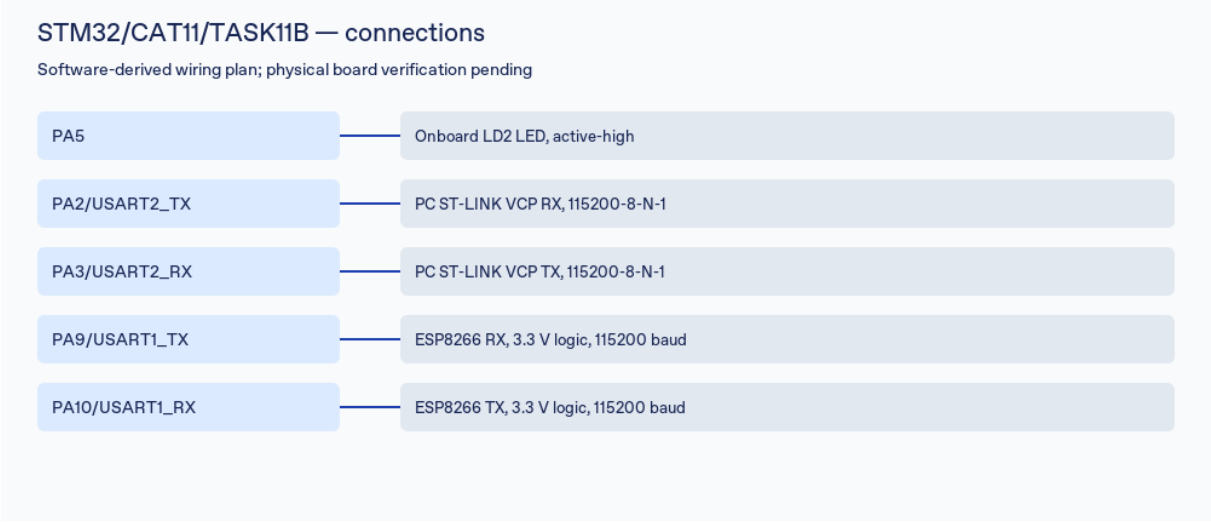
Implement and examine esp8266 http response server on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PA9/USART1_TX | ESP8266 RX, 3.3 V logic, 115200 baud |

| PA10/USART1_RX | ESP8266 TX, 3.3 V logic, 115200 baud |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

For supported modem request frames, the application emits its static HTTP response. Verify AT firmware framing and browser behavior on hardware.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT11/TASK11B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

For supported modem request frames, the application emits its static HTTP response. Verify AT firmware framing and browser behavior on hardware.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Wi-Fi credential strings are placeholders. Configure credentials locally and keep those changes uncommitted. AT-command state/error recovery and request framing remain subject to integration testing.

STM32/CAT12/TASK12A

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT12/TASK12A: NUCLEO onboard LED blink

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

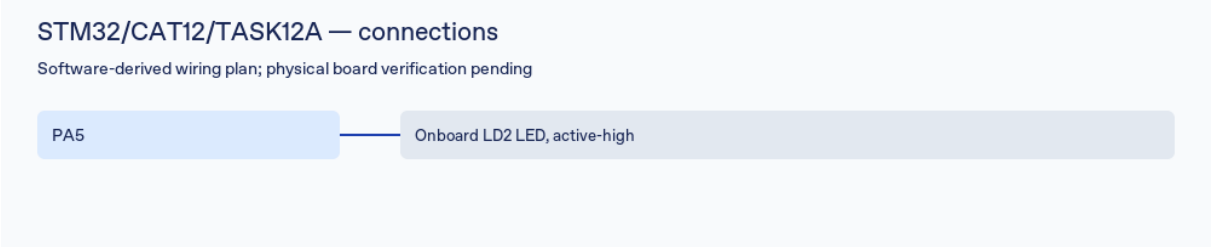
Implement and examine nucleo onboard led blink on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

The PA5 LED blinks at the programmed interval.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT12/TASK12A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

The PA5 LED blinks at the programmed interval.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT12/TASK12B

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx

- [PASS] MDK-ARM/startup_*.s

- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT12/TASK12B: NUCLEO debounced push-button toggle

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

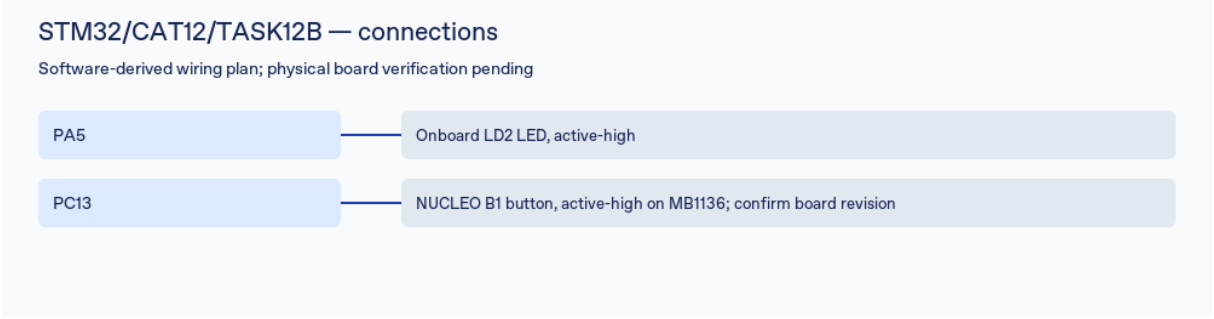
Implement and examine nucleo debounced push-button toggle on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PC13 | NUCLEO B1 button, active-high on MB1136; confirm board revision |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Each stable active-high PC13 button press toggles the PA5 LED once.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT12/TASK12B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Each stable active-high PC13 button press toggles the PA5 LED once.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT12/TASK12C

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT12/TASK12C: NUCLEO short/long press actions

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

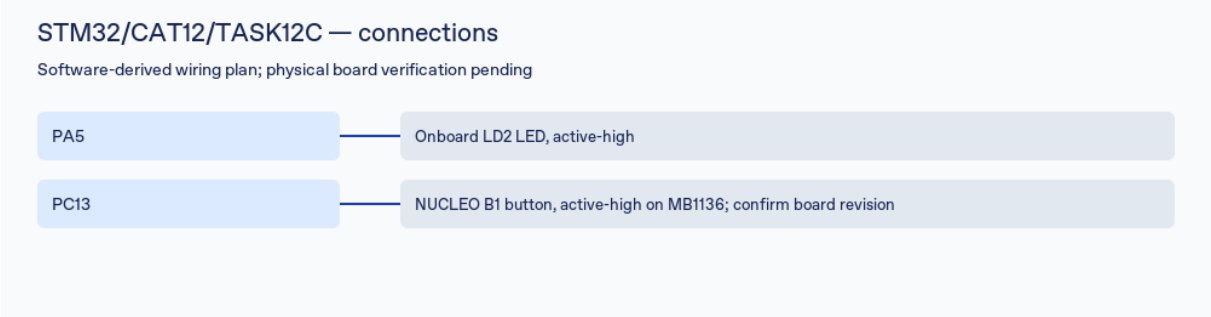
Implement and examine nucleo short/long press actions on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PC13 | NUCLEO B1 button, active-high on MB1136; confirm board revision |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

A short press toggles the PA5 LED; a long press triggers five rapid alert blinks.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT12/TASK12C

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

A short press toggles the PA5 LED; a long press triggers five rapid alert blinks.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT13/TASK13

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT13/TASK13: BME280 compensated sensor acquisition

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine bme280 compensated sensor acquisition on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

| PB8/I2C1_SCL | BME280 SCL, pull-up to 3.3 V |

| PB9/I2C1_SDA | BME280 SDA, pull-up to 3.3 V |

| 3.3V / GND | BME280 power / ground; SDO low for address 0x76 |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Real Bosch-compensated temperature, humidity and pressure JSON is sent over USART2. Sensor communication failures print an error instead of invented readings.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT13/TASK13

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Real Bosch-compensated temperature, humidity and pressure JSON is sent over USART2. Sensor communication failures print an error instead of invented readings.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

BME280 acquisition uses the original repository's bme_lab adapter and Bosch SensorAPI. The Bosch license is preserved in Core/BOSCH_LICENSE. The report's single TASK13 path is mapped from reference TASK13A; previous HTTP/Node-RED work remains recoverable from Git history.

STM32/CAT14/TASK14

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

CAT14 / TASK14: Environmental sensing and MQTT telemetry

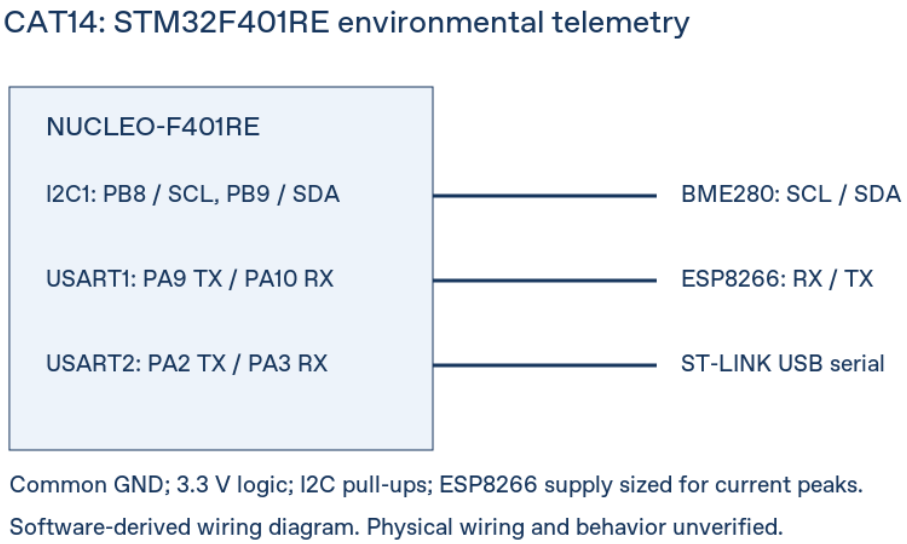
Aim and provenance

Read real BME280 measurements on STM32F401RE and publish JSON over an ESP8266 TCP/MQTT connection. Adapted from the user-designated [donor repository](https://github.com/saketh3159/25011102092_FIoT_LAB), commit 1eb9b35fc7c64b5af9bed676f3a492881984d5ef, STM32/CAT15/TASK15. The donor CAT14 skeleton did not implement its documented telemetry, so this complete project is adapted for CAT14. No assignment handout was available; full handout compliance is unverified.

Hardware and software

NUCLEO-F401RE, BME280 breakout, ESP8266 with compatible AT firmware, suitable 3.3 V supply, common ground, STM32 HAL, Bosch BME280 SensorAPI, GNU Arm GCC or Keil MDK, and Node-RED for the CAT15 dashboard.

Circuit diagram and pin connections



| MCU signal | Connection |

|---|---|

| PB8 / SCL, PB9 / SDA | BME280 I2C1, pull-ups to 3.3 V |

| PA9 / USART1 TX | ESP8266 RX, 115200 8-N-1 |

| PA10 / USART1 RX | ESP8266 TX |

| PA2 / USART2 TX | ST-LINK serial terminal, 115200 8-N-1 |

| PA3 / USART2 RX | ST-LINK serial terminal |

| PA5 | Onboard LED |

Use 3.3 V logic and a supply supporting ESP8266 current peaks. BME280 CSB is HIGH for I2C; SDO selects 0x76 or 0x77. The diagram is derived from source/MSP and CubeMX pin configuration; physical wiring is unverified.

Working principle and implementation

The application probes addresses 0x76/0x77, checks Bosch initialization and compensation settings, and reads temperature, pressure and humidity. Failed sensor reads are discarded. It sends an anonymous MQTT 3.1.1 CONNECT, checks a successful CONNACK, and publishes valid samples every five seconds at QoS 0. JSON keys are temperature (C), pressure (hPa), and humidity (%RH). Topic: fiot/25011102113/cat14/environment on broker.emqx.io:1883.

The corrected polling ESP8266 driver uses bounded timeouts, checked buffers, binary packet construction and variable MQTT Remaining Length encoding. Only clean-session anonymous CONNECT and QoS 0 PUBLISH are implemented. ESP-AT must use single-connection active receive mode with default +IPD,<length>: framing. Modem/broker failures halt with a serial message; reset after correcting the connection. Automatic reconnection, TLS and publish acknowledgments are not implemented. SEND OK confirms modem transport acceptance, not subscriber delivery.

Compile and execute

From the repository root:

sh

python3 tools/build.py STM32/CAT14/TASK14

See [build setup](../../../docs/BUILDING.md). Alternatively open MDK-ARM/IOT_project.uvprojx in Keil with STM32F4 support. Configure local Wi-Fi in Core/Inc/wifi_config.h using the included example, rebuild, flash and open the serial terminal. Keep real credentials out of Git. Subscribe to the topic above. Keil and flashing are unverified.

Expected output

Successful sensor and MQTT initialization followed by serial measurements and JSON at five-second intervals. Example schema only (not a measured result): {"temperature":25.0,"pressure":1013.25,"humidity":50.0}.

Actual verified results

Consult [fresh build evidence](../../../docs/build_results.json) and [validation notes](../../../docs/VALIDATION.md). Host tests cover MQTT serialization, rejection of oversized input, broker rejection and transport timeout. No physical measurements, hardware execution, simulator runs or live broker delivery are claimed.

References

[Attribution and licenses](../../../docs/ATTRIBUTION.md); Bosch notices are retained in Core/BOSCH_LICENSE. Existing source/vendor notices remain intact.

STM32/CAT15/TASK15

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

CAT15 / TASK15: BME280 environmental monitoring project with Node-RED

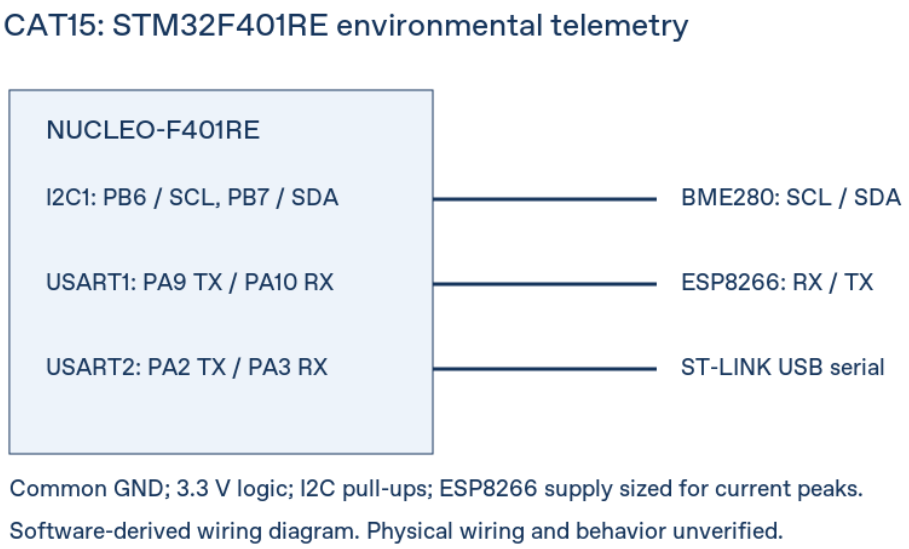
Aim and provenance

Read real BME280 measurements on STM32F401RE and publish JSON over an ESP8266 TCP/MQTT connection. Adapted from the user-designated [donor repository](https://github.com/saketh3159/25011102092_FIoT_LAB), commit 1eb9b35fc7c64b5af9bed676f3a492881984d5ef, STM32/CAT15/TASK15. The donor CAT14 skeleton did not implement its documented telemetry, so this complete project is adapted for CAT14. No assignment handout was available; full handout compliance is unverified.

Hardware and software

NUCLEO-F401RE, BME280 breakout, ESP8266 with compatible AT firmware, suitable 3.3 V supply, common ground, STM32 HAL, Bosch BME280 SensorAPI, GNU Arm GCC or Keil MDK, and Node-RED for the CAT15 dashboard.

Circuit diagram and pin connections



| MCU signal | Connection |

|---|---|

| PB6 / SCL, PB7 / SDA | BME280 I2C1, pull-ups to 3.3 V |

| PA9 / USART1 TX | ESP8266 RX, 115200 8-N-1 |

| PA10 / USART1 RX | ESP8266 TX |

| PA2 / USART2 TX | ST-LINK serial terminal, 115200 8-N-1 |

| PA3 / USART2 RX | ST-LINK serial terminal |

| PA5 | Onboard LED |

Use 3.3 V logic and a supply supporting ESP8266 current peaks. BME280 CSB is HIGH for I2C; SDO selects 0x76 or 0x77. The diagram is derived from source/MSP and CubeMX pin configuration; physical wiring is unverified.

Working principle and implementation

The application probes addresses 0x76/0x77, checks Bosch initialization and compensation settings, and reads temperature, pressure and humidity. Failed sensor reads are discarded. It sends an anonymous MQTT 3.1.1 CONNECT, checks a successful CONNACK, and publishes valid samples every five seconds at QoS 0. JSON keys are temperature (C), pressure (hPa), and humidity (%RH). Topic: fiot/25011102113/cat15/environment on broker.emqx.io:1883.

The corrected polling ESP8266 driver uses bounded timeouts, checked buffers, binary packet construction and variable MQTT Remaining Length encoding. Only clean-session anonymous CONNECT and QoS 0 PUBLISH are implemented. ESP-AT must use single-connection active receive mode with default +IPD,<length>: framing. Modem/broker failures halt with a serial message; reset after correcting the connection. Automatic reconnection, TLS and publish acknowledgments are not implemented. SEND OK confirms modem transport acceptance, not subscriber delivery.

Compile and execute

From the repository root:

sh

python3 tools/build.py STM32/CAT15/TASK15

See [build setup](../../../docs/BUILDING.md). Alternatively open MDK-ARM/IOT_project.uvprojx in Keil with STM32F4 support. Configure local Wi-Fi in Core/Inc/wifi_config.h using the included example, rebuild, flash and open the serial terminal. Keep real credentials out of Git. Subscribe to the topic above. Keil and flashing are unverified.

Expected output

Successful sensor and MQTT initialization followed by serial measurements and JSON at five-second intervals. Example schema only (not a measured result): {"temperature":25.0,"pressure":1013.25,"humidity":50.0}.

Actual verified results

Consult [fresh build evidence](../../../docs/build_results.json) and [validation notes](../../../docs/VALIDATION.md). Host tests cover MQTT serialization, rejection of oversized input, broker rejection and transport timeout. No physical measurements, hardware execution, simulator runs or live broker delivery are claimed.

References

[Attribution and licenses](../../../docs/ATTRIBUTION.md); Bosch notices are retained in Core/BOSCH_LICENSE. Existing source/vendor notices remain intact.

STM32/CAT8/TASK8A

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT8/TASK8A: Blue Pill GPIO button follow

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

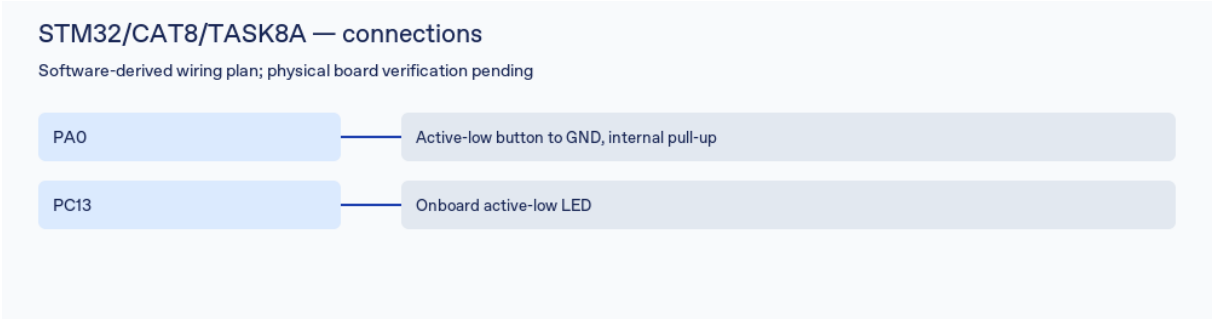
Implement and examine blue pill gpio button follow on STM32F103C8T6 Blue Pill.

Components required and hardware platform

STM32F103C8T6 Blue Pill board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA0 | Active-low button to GND, internal pull-up |

| PC13 | Onboard active-low LED |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

The onboard PC13 LED follows the active-low PA0 button.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT8/TASK8A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

The onboard PC13 LED follows the active-low PA0 button.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for

this repository.

STM32/CAT8/TASK8B

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT8/TASK8B: Blue Pill debounced LED toggle

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

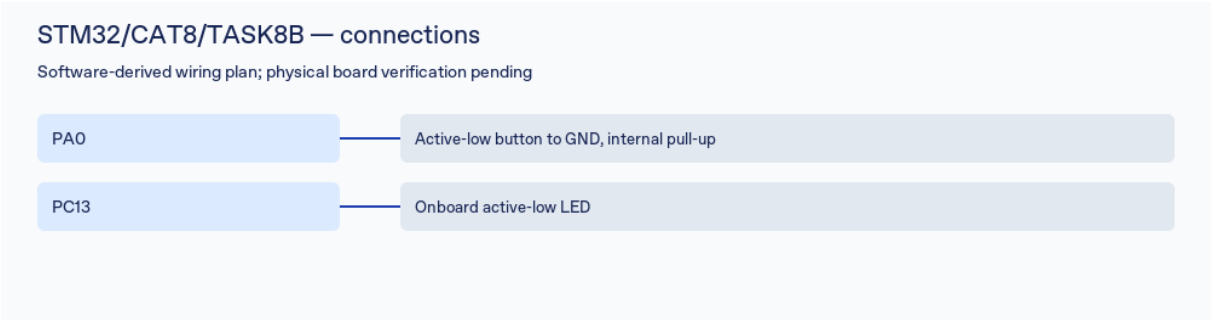
Implement and examine blue pill debounced led toggle on STM32F103C8T6 Blue Pill.

Components required and hardware platform

STM32F103C8T6 Blue Pill board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA0 | Active-low button to GND, internal pull-up |

| PC13 | Onboard active-low LED |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Each stable active-low PA0 press toggles the PC13 LED.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT8/TASK8B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Each stable active-low PA0 press toggles the PC13 LED.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT8/TASK8C

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT8/TASK8C: Blue Pill button press classification

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

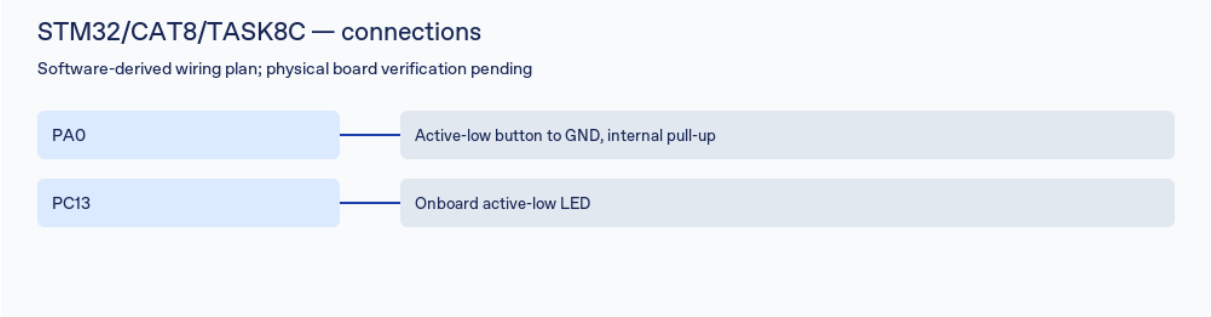
Implement and examine blue pill button press classification on STM32F103C8T6 Blue Pill.

Components required and hardware platform

STM32F103C8T6 Blue Pill board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA0 | Active-low button to GND, internal pull-up |

| PC13 | Onboard active-low LED |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

The application distinguishes short and long presses using the implemented threshold.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT8/TASK8C

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

The application distinguishes short and long presses using the implemented threshold.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT9/TASK9A

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT9/TASK9A: NUCLEO UART character echo

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

Implement and examine nucleo uart character echo on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Received USART2 characters are echoed to the PC terminal.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT9/TASK9A

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Received USART2 characters are echoed to the PC terminal.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

STM32/CAT9/TASK9B

File checks

- [PASS] .ioc file
- [PASS] .mxproject
- [PASS] Core/ folder
- [PASS] Drivers/ folder
- [PASS] MDK-ARM/ folder
- [PASS] MDK-ARM/*.uvprojx
- [PASS] MDK-ARM/startup_*.s
- [PASS] README.md exists.

Task result: PASSED

README content

STM32/CAT9/TASK9B: NUCLEO UART LED command parser

Student: LAKSHMI KARTHIK VEMULA · Registration: 25011102113

Course: Fundamentals of IoT Laboratory · Shiv Nadar University Chennai, Computer Science and Engineering

Aim

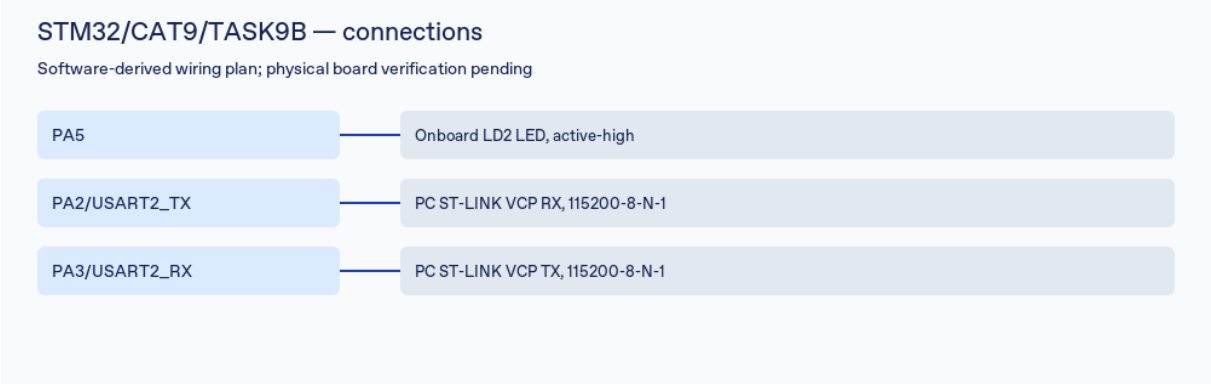
Implement and examine nucleo uart led command parser on STM32F401RET6 NUCLEO-F401RE.

Components required and hardware platform

STM32F401RET6 NUCLEO-F401RE board, USB/ST-LINK debugger, serial terminal, and the peripherals listed below.

Use board power and common GND. STM32 GPIO signals use 3.3 V logic. Verify supply/current requirements and voltage compatibility before connecting peripherals.

Circuit diagram



This schematic records the configured software pins. Actual board wiring, component ratings and timing have NOT VERIFIED status.

Pin connections

| MCU signal | Connection |

|---|---|

| PA5 | Onboard LD2 LED, active-high |

| PA2/USART2_TX | PC ST-LINK VCP RX, 115200-8-N-1 |

| PA3/USART2_RX | PC ST-LINK VCP TX, 115200-8-N-1 |

For an ESP8266, use a supply capable of its peak current and compatible AT-command firmware. Hardware UART bridges, modem responses and sensor readings require board testing.

Working principle and implementation

Terminal LED commands change or query the PA5 onboard LED.

Read the complete [application source](Core/Src/main.c) for the implemented state transitions, parser, delays and peripheral initialization. Documentation follows the actual code, rather than conflicting titles in the source reference.

Compilation instructions

From the repository root, run:

sh

python3 tools/build.py STM32/CAT9/TASK9B

See [toolchain setup and build limits](../../../docs/BUILDING.md). Fresh firmware is written to Objects/firmware.hex. The GNU build compiles the project sources, assembles the device vector table through GNU startup, links firmware, and emits Intel HEX. Keil MDK and CubeMX regeneration have not been run.

Expected output

Terminal LED commands change or query the PA5 onboard LED.

Actual verified results

See [recorded build and file checks](../../../docs/VALIDATION.md) for the result and artifact hash. A successful compile/link does not verify timing, peripherals or assignment behavior. Hardware execution, oscilloscope measurements and simulator runs are NOT VERIFIED.

References and attribution

This task contains reference-derived source/project material, with documented corrections. It is not represented as solely student-authored code or as independently completed physical laboratory work. See [provenance and third-party notices](../../../docs/ATTRIBUTION.md). The circuit SVG and this task documentation were generated for this repository.

Overall Summary

- Required CAT folders: 14
- Projects discovered: 29
- Projects passed: 29
- Projects failed: 0
- File checks passed: 206/206

Required CAT folder validation

- [FOUND] 8051/CAT2
- [FOUND] 8051/CAT3
- [FOUND] 8051/CAT4
- [FOUND] 8051/CAT5
- [FOUND] 8051/CAT6
- [FOUND] 8051/CAT7
- [FOUND] STM32/CAT8
- [FOUND] STM32/CAT9
- [FOUND] STM32/CAT10
- [FOUND] STM32/CAT11
- [FOUND] STM32/CAT12
- [FOUND] STM32/CAT13
- [FOUND] STM32/CAT14
- [FOUND] STM32/CAT15

Overall Status: PASSED